

Búsqueda de hiperparámetros y TP1

Clase práctica 6

Motivación

- ¿Cómo podemos encontrar los hiperparámetros que **optimizan** nuestro modelo?
- ¿Cuáles son las **ventajas y desventajas** de esas estrategias?
- ¿Cómo venimos con el **TP1**? ¿Tenemos alguna duda?

Organización

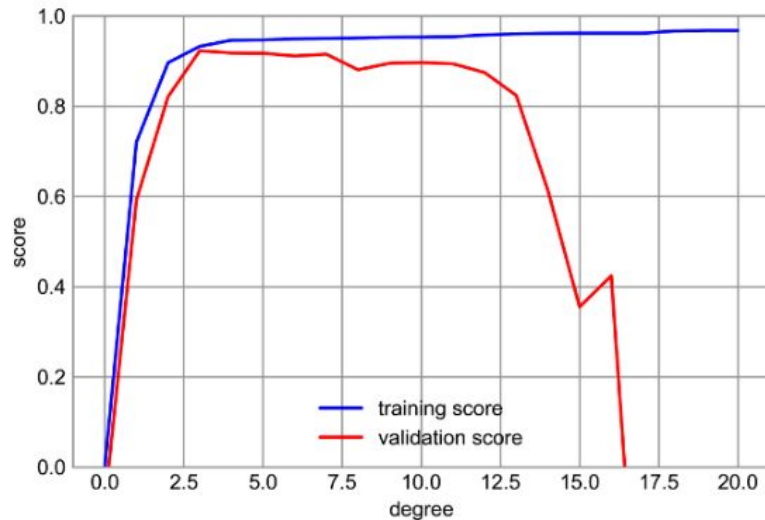
Etapa	Inicio		Duración
	Sección 1	Sección 2	
Introducción	08:00	09:50	05'
<i>Grid vs. random search</i>	08:05	09:55	30'
<i>Bayesian optimization</i>	08:35	10:25	20'
TP1 Avances y consultas	08:55	10:45	40'
Cierre	09:35	11:25	05'
Fin de la clase	09:40	11:30	-

Enfoque hasta ahora | Un hiperparámetro

Ejemplo: ajuste mediante **polinomio**.

¿Qué **grado** nos conviene usar?

Hasta ahora, nuestro enfoque fue **elegir valores y probarlos** para el hiperparámetro y analizar la *performance* del modelo en validación.

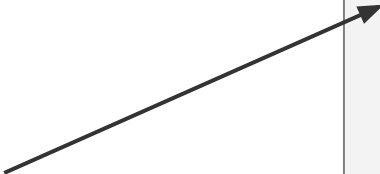


Enfoque hasta ahora | N hiperparámetros

Queremos encontrar los valores de esos hiperparámetros que optimicen el modelo en validación.

¿Cómo lo venimos haciendo intuitivamente?

¡Se llama **grid search**!

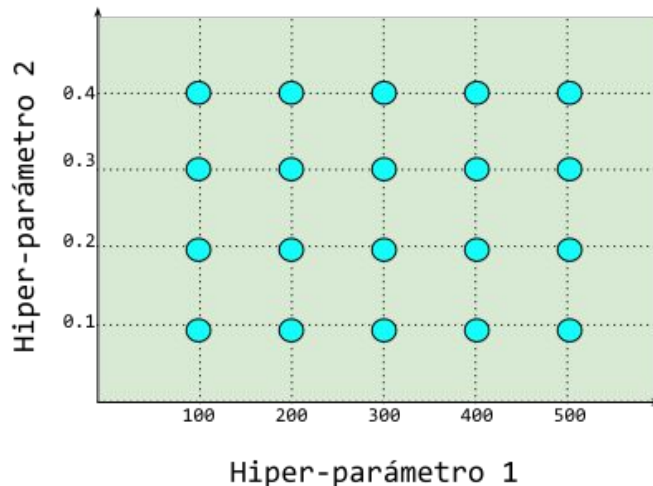


```
for val_1 in valores_hip_1:
    ...
    for val_N in valores_hip_N:
        train(val_1 , ..., val_N)
        calcular_score()

        if score mejoró:
            guardar_valores()
```

Grid search

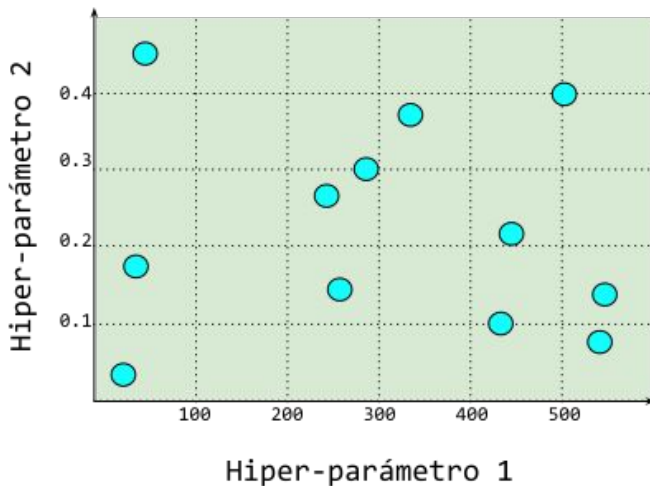
- Dado un conjunto de hiperparámetros y posibles valores seleccionados, se prueban todas las **combinaciones**.
- Se definen **explícitamente** los posibles valores de cada hiperparámetro.
- Requiere **mucho cómputo**.



```
sklearn.model_selection.GridSearchCV()
```

Random search

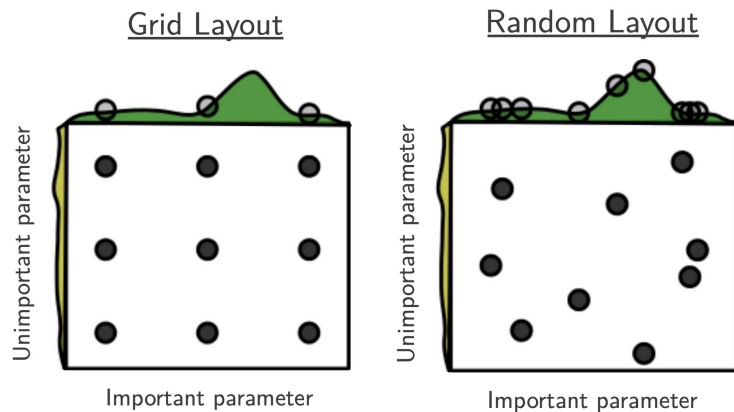
- Dado un espacio de hiperparámetros definido, se toman **combinaciones aleatorias**.
- Se definen valores a probar de los hiperparámetros o **distribuciones** de las cuales se seleccionarán los mismos (útil para valores continuos).
- Requiere **menos cómputo**.
- Se define el número de **iteraciones** máximas.
- **No tiene memoria** de la performance de iteraciones pasadas (grid tampoco).



```
sklearn.model_selection.RandomizedSearchCV()
```

¿Cuál nos conviene usar?

- Evidencia empírica muestra que **random search** encuentra configuraciones iguales o, a veces, incluso mejores que grid search, en una fracción del tiempo.
- Random search permite **explorar** el espacio de hiperparámetros de manera **más eficiente**.
- En consecuencia, si se asigna el **mismo tiempo** de cómputo, random search encuentra **mejores configuraciones**.



Grid y random search | Modo

Conozcamos las implementaciones de grid search y random search en scikit-learn. Para ello, veamos `td6-p06-c-grid-random.ipynb`.

Grid y random search | Documentación oficial

- `sklearn.model_selection.GridSearchCV()`: [enlace](#).
- `sklearn.model_selection.RandomizedSearchCV()`: [enlace](#).

¿Se pueden mejorar? 🤔

Con grid y random search, a medida que se exploran los hiperparámetros, se encuentran puntos del espacio que generan distintos resultados. Sin embargo, esas dos estrategias **no aprovechan el conocimiento que se va adquiriendo** a medida que se prueban distintos valores.

Bayesian optimization | Idea



Tratar la búsqueda de hiperparámetros como un **problema de machine learning** en sí mismo.

- Asumamos que existe una **función** f que, para cada **vector de hiperparámetros** x , devuelve la **performance estimada del modelo** (p).
- Desconocemos la forma de f , pero la podemos **estimar** (si bien es caro hacerlo).
- Cuando **probamos un nuevo valor** de x , adquirimos más información que permite estimar mejor f (es decir, adquirimos un dato más para estimar f).
- Dada una estimación de f , ¿en **dónde** probamos un nuevo x ? Dos objetivos:
 - **Exploration**: probar nuevos y diversos valores de x para estimar mejor f **en todo el espacio**; el óptimo quizás está en una zona aún no explorada.
 - **Exploitation**: dada nuestra estimación de f en un momento, probar valores **cercanos al óptimo provisorio**, de acuerdo a nuestra estimación actual, e intentar aprender más de f en esa área prometedora.

Optimización bayesiana **balancea** el *trade-off* entre esos dos objetivos, con el fin de **optimizar** p .

Bayesian optimization | Modo

Conozcamos una implementación de bayesian optimization con la librería **HyperOpt**.
Para ello, veamos `td6-p06-d-bayesian.ipynb`.

Bayesian optimization | Documentación oficial

- Librería **HyperOpt**: [enlace](#).
- **scikit-optimize** como librería alternativa: [enlace](#).
 - Bastante utilizada, **pero** dejó de recibir actualizaciones hace 2 años.

TP1 | Avances y consultas

El tiempo que nos queda de la clase de hoy es para que cada grupo pueda **avanzar** en su resolución del TP1 y **consultar** a los auxiliares docentes las dudas que tenga.

Cierre

Hoy vimos, entre otras cuestiones:

- cómo podemos encontrar los hiperparámetros que **optimizan** nuestro modelo,
- cuáles son las **ventajas y desventajas** de esas estrategias y
- cuáles son las respuestas a nuestras dudas del **TP1**.

Pueden darnos *feedback* de la clase [acá](#).